

1-

a)



The screenshot shows the OneCompiler Java IDE. The code in Main.java is as follows:

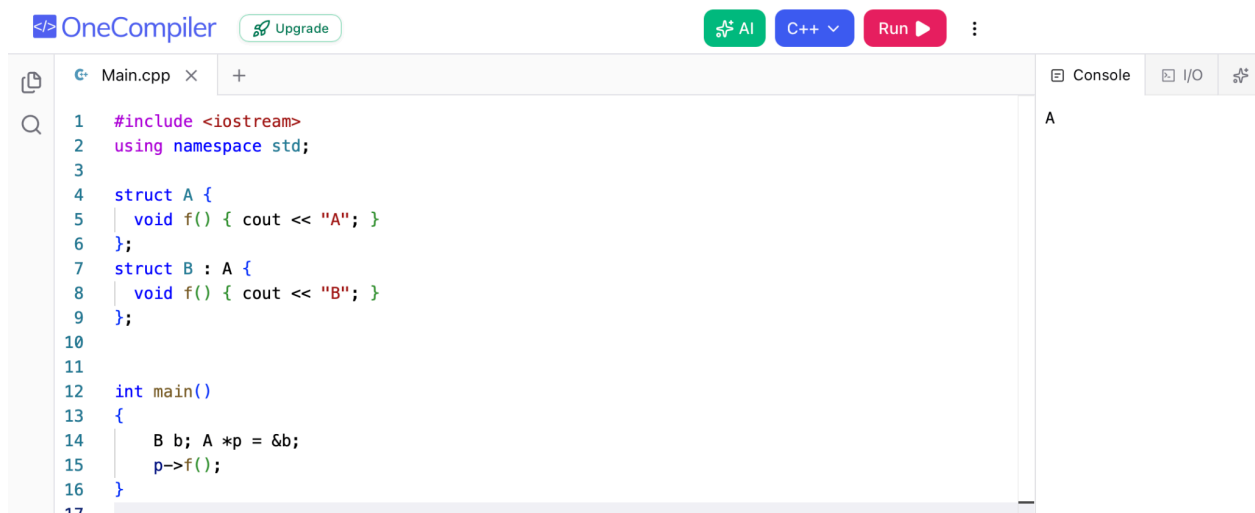
```
1 import java.util.*;
2
3 class Animal {
4     String som() { return "..."; }
5 }
6 class Cachorro extends Animal {
7     String som() { return "au"; }
8 }
9
10 public class Main {
11     public static void main(String[] args) {
12         Animal x = new Cachorro();
13         System.out.println(x.som());
14     }
15 }
16
```

The console output on the right is "au".

b) Java. `som()` é sobrescrito e `x` aponta para um `Cachorro`. O despacho dinâmico escolhe o método pelo tipo do objeto, e não pelo tipo da variável (`Animal`), então sai `au`.

2-

a)



The screenshot shows the OneCompiler C++ IDE. The code in Main.cpp is as follows:

```
1 #include <iostream>
2 using namespace std;
3
4 struct A {
5     void f() { cout << "A"; }
6 };
7 struct B : A {
8     void f() { cout << "B"; }
9 };
10
11
12 int main()
13 {
14     B b; A *p = &b;
15     p->f();
16 }
17
```

The console output on the right is "A".

b) C++. `f()` não é virtual, então a vinculação é estática e usa o tipo do ponteiro (`A*`). Sai `A`, mesmo que o objeto seja um `B`. Com `virtual` em `A::f`, sairia `B`.

3-

a)

The screenshot shows the OneCompiler Java IDE. The code in Main.java is as follows:

```
1 import java.util.*;
2
3 class A { String nome = "A";
4     String getNome() { return nome; } }
5 class B extends A { String nome = "B";
6     String getNome() { return nome; } }
7
8
9 public class Main {
10     public static void main(String[] args) {
11         A x = new B();
12         System.out.println(x.nome + " " + x.getNome());
13     }
14 }
15
```

The console output on the right is "A B".

b) Java. Campos não são polimórficos, só métodos.

- `x.nome` é resolvido pelo tipo declarado (A) e dá "A".
- `x.getNome()` é despachado dinamicamente para `B.getNome()`, que devolve o nome de B, ou seja, "B".

4-

a)

The screenshot shows the OneCompiler Python IDE. The code in main.py is as follows:

```
1 class Contador:
2     total = 0
3     def __init__(self):
4         Contador.total += 1
5         self.id = Contador.total
6
7 a = Contador(); b = Contador()
8 print(a.id, b.id, a.total)
9
```

The console output on the right is "1 2 2".

b) Python. `total` é um atributo da classe, compartilhado por todas as instâncias. Cada `__init__` incrementa `Contador.total` e copia o valor para `self.id`, então `a.id = 1` e `b.id = 2`. `a.total` busca o atributo na classe e vale 2. Em Python, toda busca de atributo e método acontece em execução.

5-

a)

```
1 import java.util.*;
2
3 class A {
4     static String quem() { return "A"; }
5 }
6 class B extends A {
7     static String quem() { return "B"; }
8 }
9
10 public class Main {
11     public static void main(String[] args) {
12         A x = new B();
13         System.out.println(x.quem());
14     }
15 }
16
```

Console: A

b) Java. Métodos static pertencem à classe e não participam do despacho dinâmico (slide 17: dinâmico em Java é tudo, exceto static, private e final). B.quem() apenas esconde A.quem(). Como o tipo declarado de x é A, sai A.

6-

a)

```
1 package main
2 import "fmt"
3
4 type Animal struct{}
5 func (Animal) Som() string { return "... " }
6 func (a Animal) Falar() string {
7     return "faz " + a.Som()
8 }
9 type Cao struct{ Animal }
10 func (Cao) Som() string { return "au" }
11
12
13
14
15 func main() {
16     fmt.Println(Cao{}.Falar(), Cao{}.Som())
17 }
```

Console: faz ... au

b) Go. Falar é um método promovido de Animal e tem receptor Animal. Dentro dele a.Som() chama Animal.Som(), e não o Som de Cao. Por isso Cao{}.Falar() dá faz..., enquanto Cao{}.Som() chama diretamente o método de Cao e dá au. Embedding é composição, não herança, e não existe vinculação dinâmica entre o tipo embutido e o que o embute.

